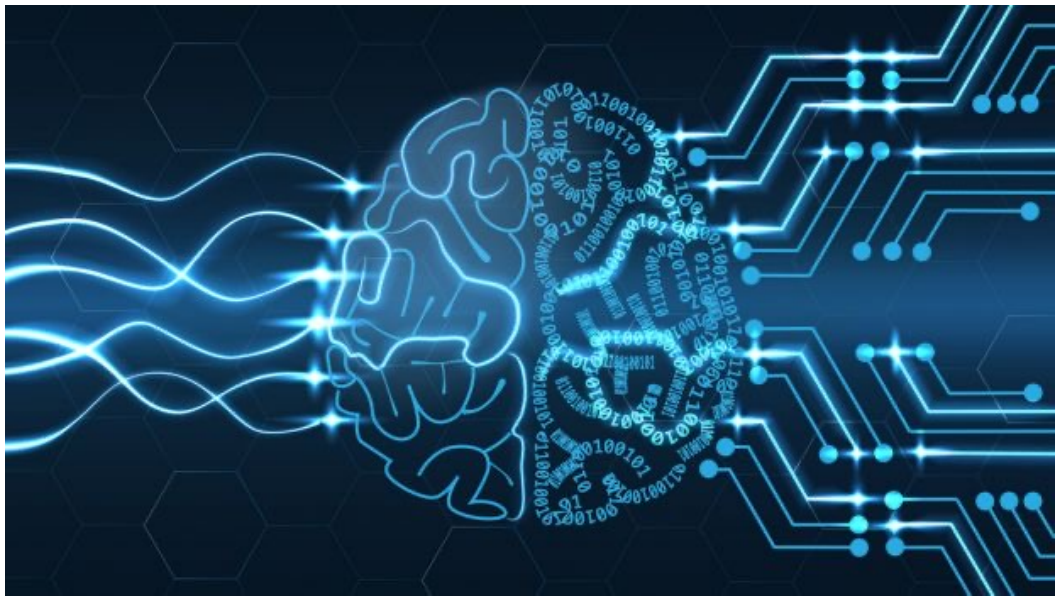


Chapter 2. Machine Learning as Function-fitting Paradigm

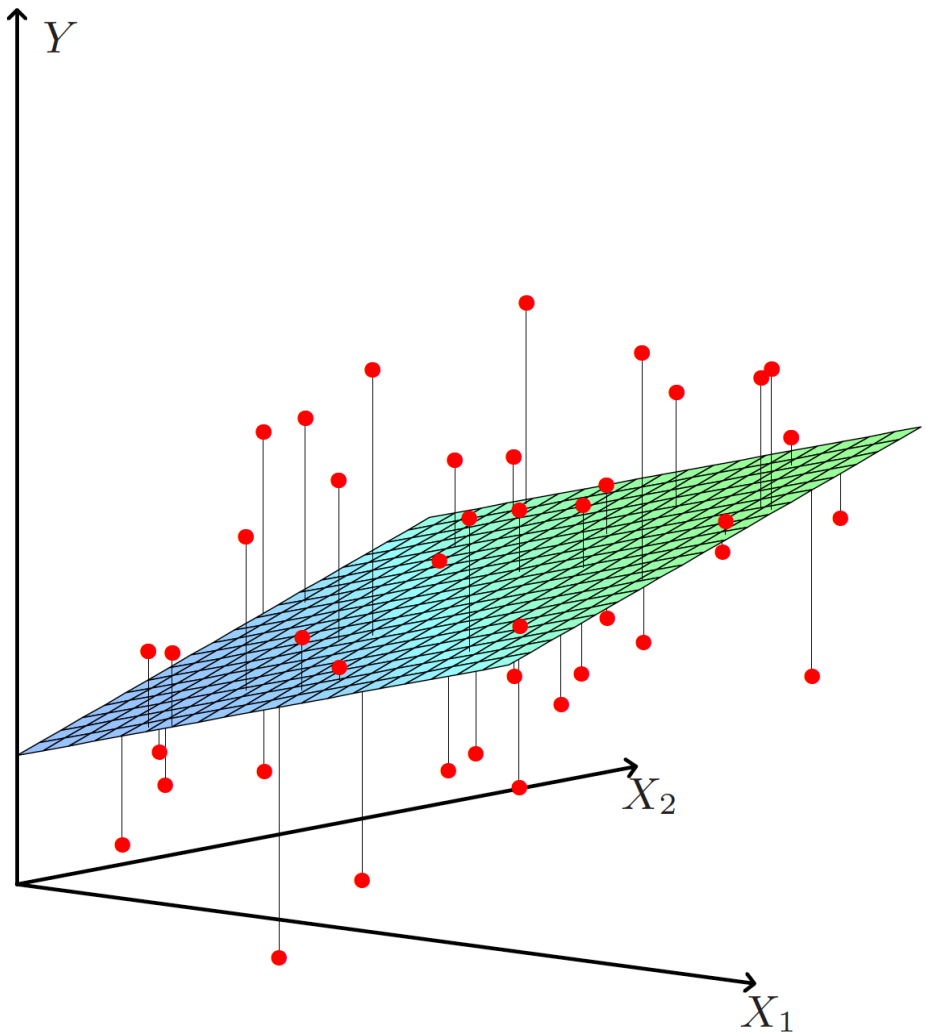
Changcheng Huang

Department of Systems and Computer Engineering

Outline

- Get started with models
- How to choose models

Linear Regression in Concept



Linear Regression in Math

- The model family

$$\hat{Y} = f(X) = \beta_0 + \sum_{j=1}^{D} X_j \beta_j$$
$$= \sum_{j=0}^{D} X_j \beta_j = \boldsymbol{X}^T \boldsymbol{\beta}\ , \qquad X_0 = 1,\ N = D + 1$$

- Training
  - The loss function

a row in

$$L(\boldsymbol{\beta}) = \frac{1}{M}\left[\sum_{i=1}^{M}\left(y_i - f\left(\boldsymbol{x}^{(i)}\right)\right)^2\right] = \frac{1}{M}\left[\sum_{i=1}^{M}\left(y_i - \boldsymbol{x}^{(i)T}\boldsymbol{\beta}\right)^2\right] = \frac{1}{M}(\boldsymbol{y} - X\boldsymbol{\beta})^T(\boldsymbol{y} - X\boldsymbol{\beta})$$

quadratic function

  - Least squares estimates

$$\nabla_\beta L(\boldsymbol{\beta}) = -2X^{\boldsymbol{T}}(\boldsymbol{y} - X\boldsymbol{\beta}) = \boldsymbol{0}$$
$$\widehat{\boldsymbol{\beta}} = (X^T X)^{-1} X^T \boldsymbol{y}$$

$O(MN^2)$ $\quad$ $O(N^3)$ $\rightarrow$ $O(NM)$

- Prediction

$$\hat{Y} = \boldsymbol{X}^T \widehat{\boldsymbol{\beta}} = \boldsymbol{X}^T (X^T X)^{-1} X^T \boldsymbol{y}$$

  - Error

$$E = \left\| Y - \hat{Y} \right\|_2^2$$

An Example

- Given the following dataset

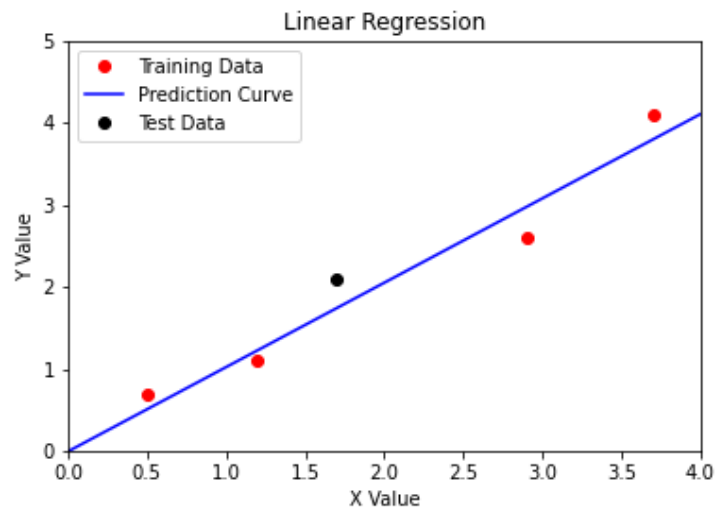
x	0.5	1.2	1.7	2.9	3.7
y	0.7	1.1	2.1	2.6	4.1

- Training

$$X = \begin{pmatrix} 1.0 & 1.0 & 1.0 & 1.0 \\ 0.5 & 1.2 & 2.9 & 3.7 \end{pmatrix}^T$$

$$\boldsymbol{y} = (0.7 \quad 1.1 \quad 2.6 \quad 4.1)^T$$

$$\hat{\beta} \approx (-0.005 \quad 1.03)^T$$



- Testing

$$\boldsymbol{x} = (1.0 \quad 1.7)^T$$

$$\hat{Y} \approx 1.74$$

$$\text{SE} \approx 0.13$$

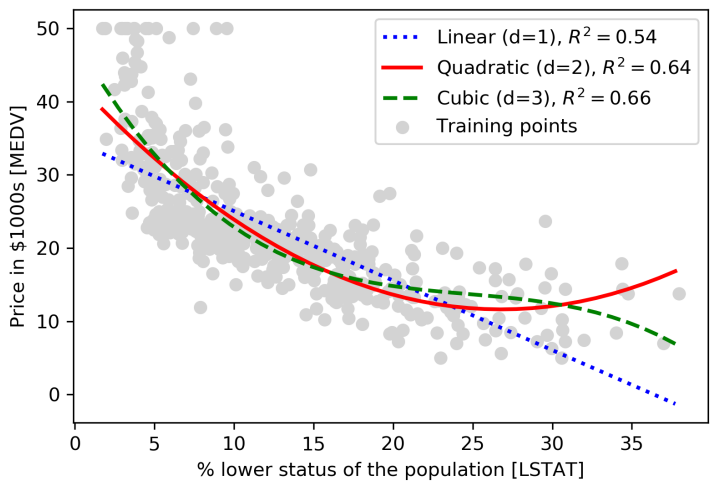
Basis Expansion

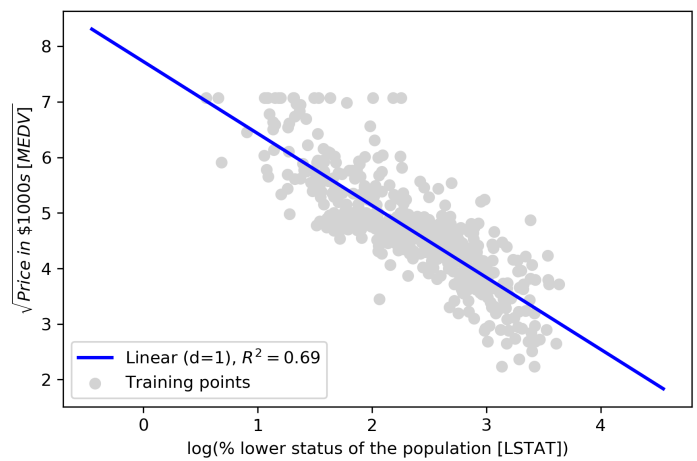
- Linear Basis Expansion

$$\hat{f}(X) = \beta_0 + \sum_{j=1}^{D} h_j(X)\beta_j = \sum_{j=0}^{D} h_j(X)\beta_j$$

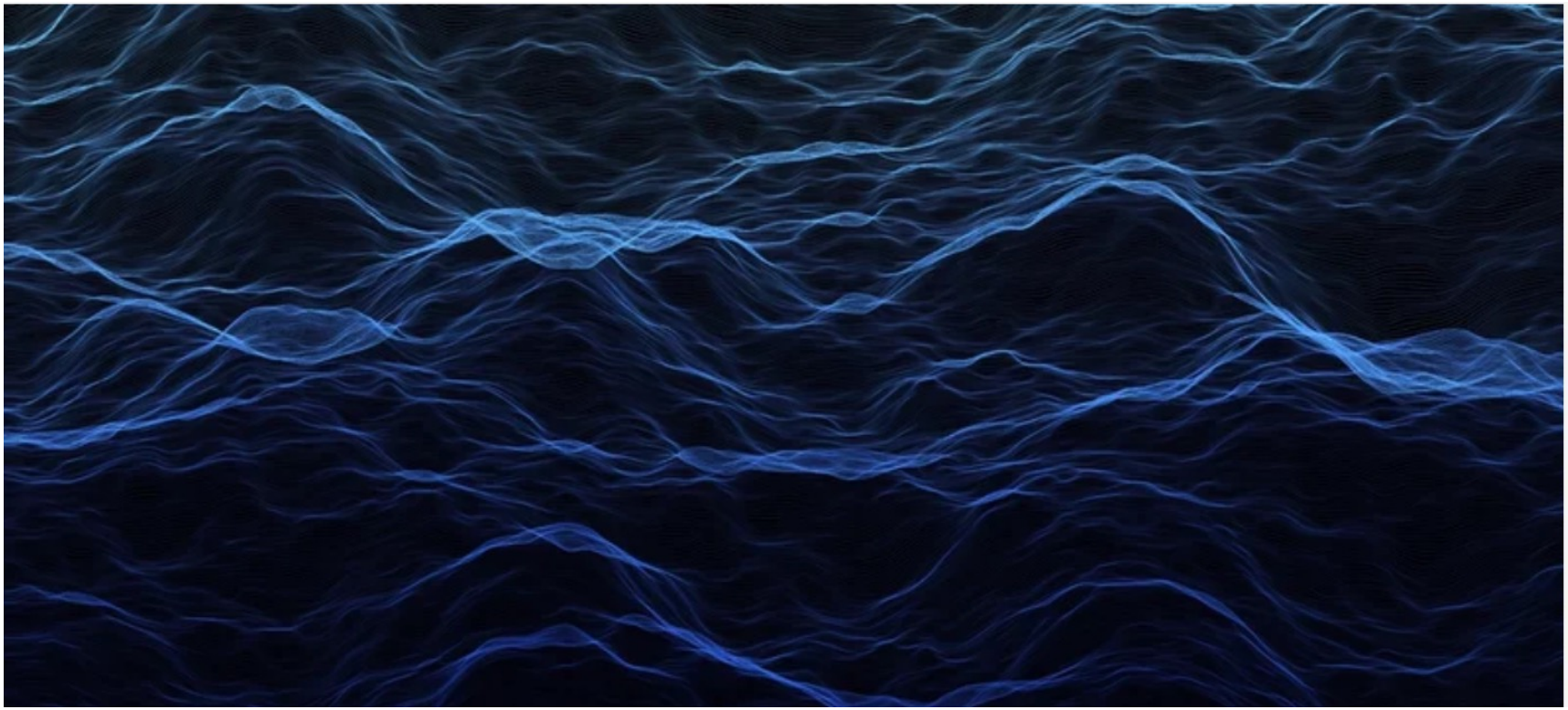
  - e.g., $h_j(X) = X_j, X_q X_r, X_q^l, \log(X_q), \cdots$

- Housing dataset example
  - Houses in the suburbs of Boston
  - 506 examples





Optimization in General



Gradient Descent

- Consider an arbitrary function

$$f: \mathbb{R}^N \to \mathbb{R}, \qquad g(\boldsymbol{x}) = \nabla_{\boldsymbol{x}} f(\boldsymbol{x}),$$

$$H_{ij}(\boldsymbol{x}) = \frac{\partial^2}{\partial x_i \partial x_j} f(\boldsymbol{x}), H = [H_{ij}]$$

- Gradient decent

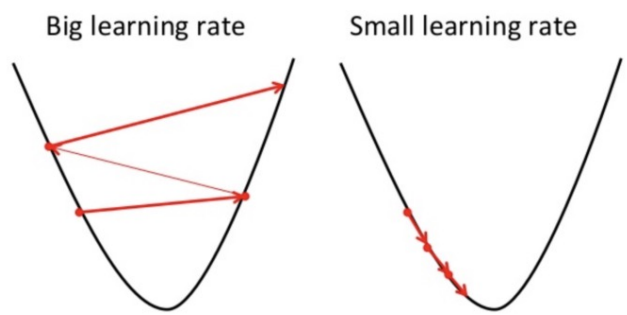
$$\boldsymbol{x}' = \boldsymbol{x} - \epsilon \nabla_{\boldsymbol{x}} f(\boldsymbol{x})$$

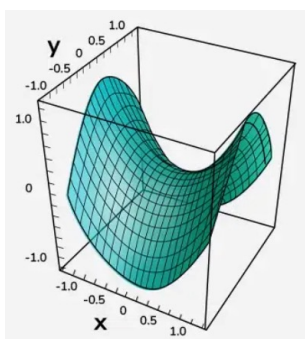
vector

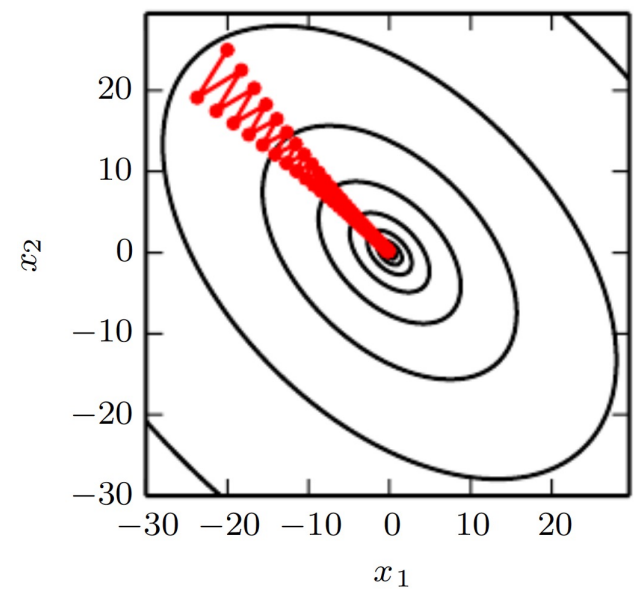
- How to decide ϵ

$$f(\boldsymbol{x} - \epsilon g) \approx f(x) - \epsilon g^T g + \frac{1}{2}\epsilon^2 g^T H g$$

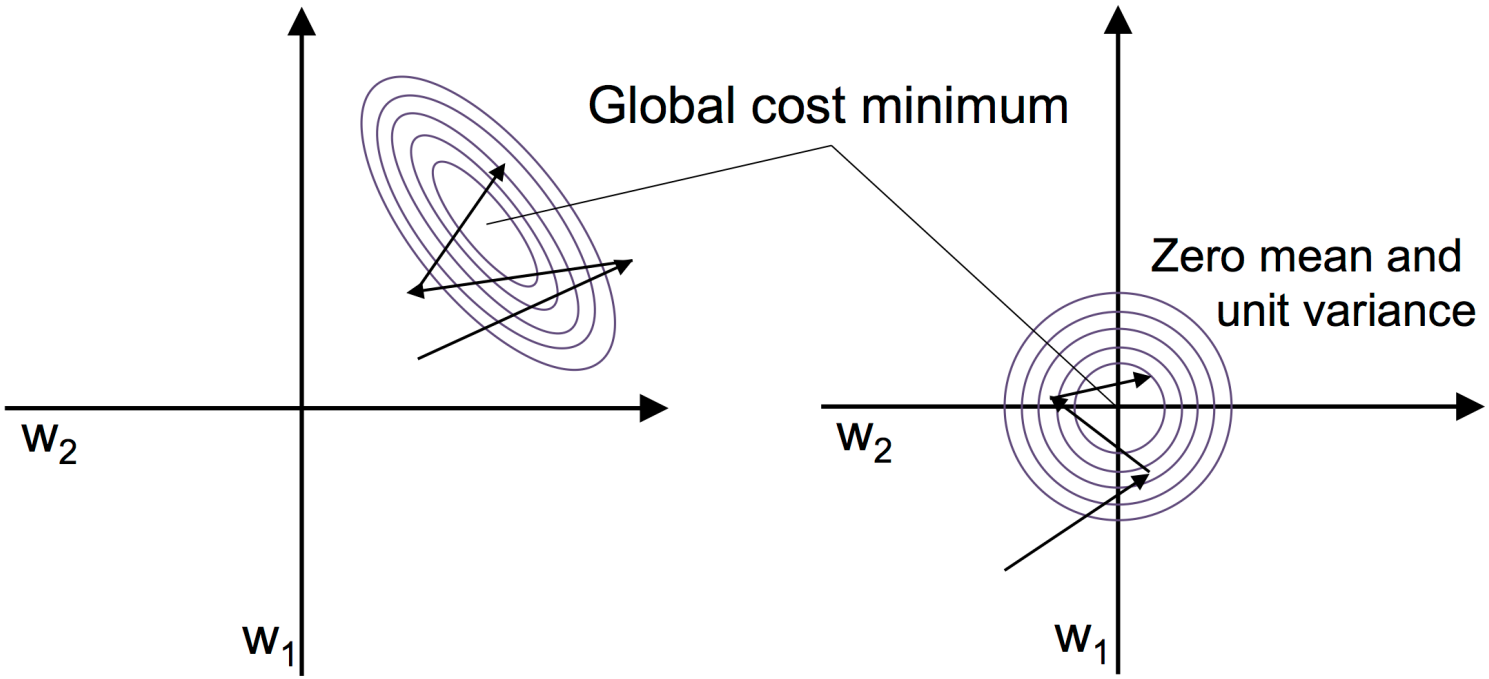
To minimize $f(x - \epsilon g)$, $\epsilon^* = \frac{g^T g}{g^T H g}$







Feature Scaling



$$x_{std}^{(i)} = \frac{x^{(i)} - \mu_x}{\sigma_x}$$

$$x_{norm}^{(i)} = \frac{x^{(i)} - x_{min}}{x_{max} - x_{min}}$$

GD for Linear Regression

- The gradient

$$\nabla_\beta L(\beta) = -2X^{\boldsymbol{T}}(\boldsymbol{y} - X\beta)$$

$O(MD)$

$$\frac{\partial L(\beta)}{\partial \beta_j} = -2\sum_{i=1}^{M} x_j^{(i)} \left(y_i - \sum_{k=0}^{D} x_k^{(i)} \beta_k\right)$$

prediction error

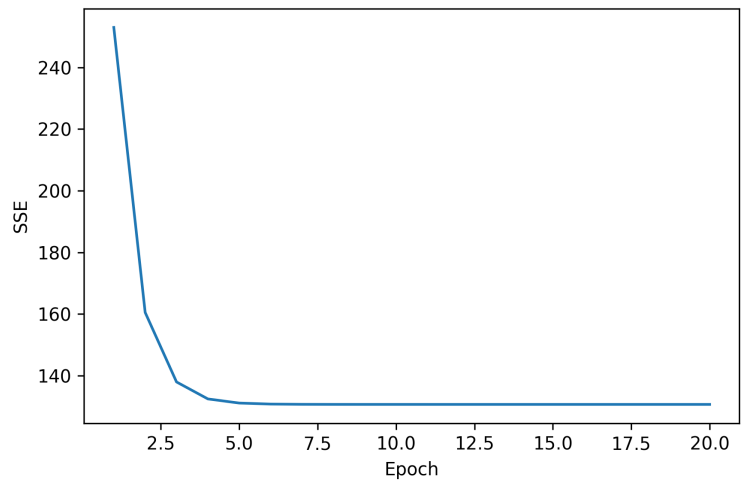
$$\beta'_j = \beta_j - \epsilon \frac{\partial L(\beta)}{\partial \beta_j}$$

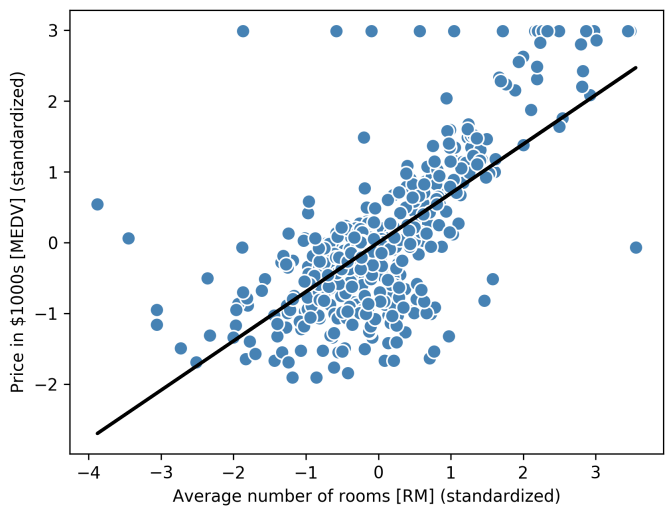
$$\beta' = \beta - \epsilon \nabla_\beta L(\beta)$$

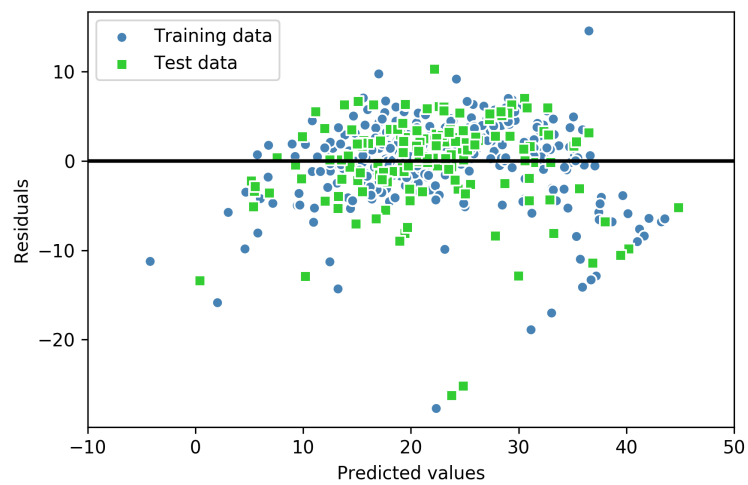
  - Backward error propagation
- A step is also called an epoch
  - Use all the training data once

An Example

- The housing dataset
  - Houses in the suburbs of Boston
  - 506 examples







Stochastic Gradient Descent (SGD)

- Work for large dataset

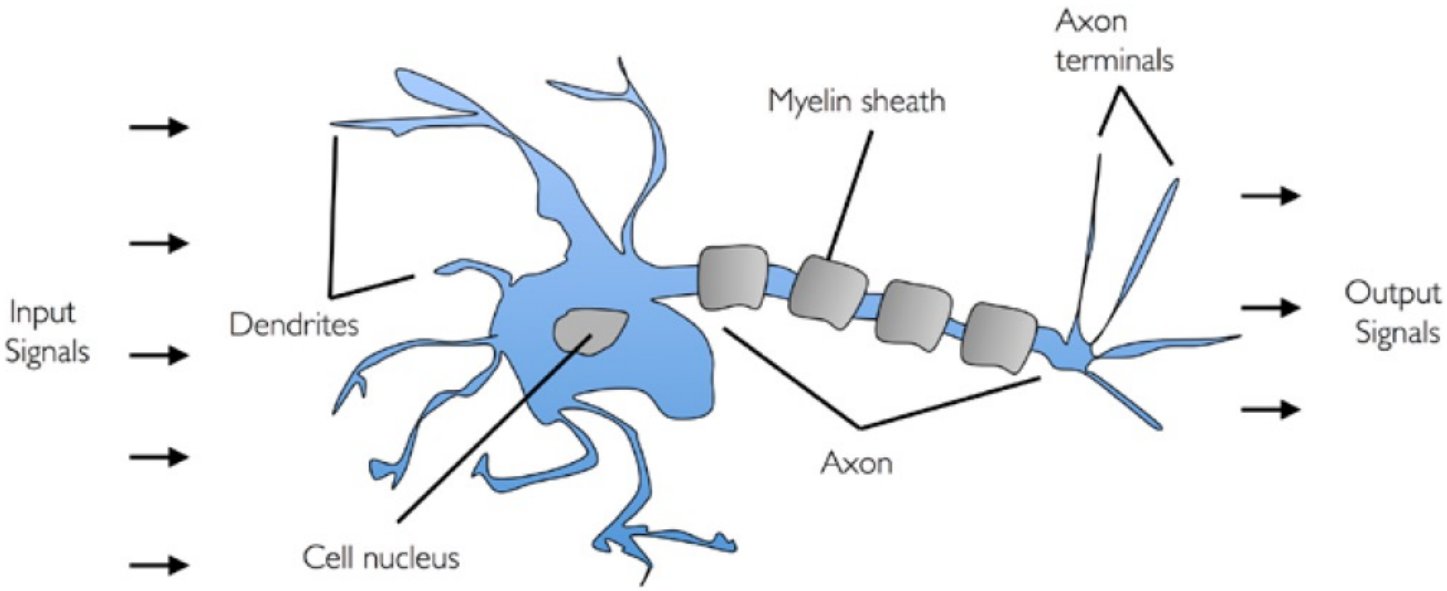
$$\nabla_\beta L(\beta) = -2X^{\boldsymbol{T}}(\boldsymbol{y} - X\beta)$$

$$\frac{\partial L(\beta)}{\partial \beta_j} = -2\sum_{i=1}^{M} x_j^{(i)}\,(y_i - \sum_{j=0}^{D} x_j^{(i)}\beta_j)$$

- For each epoch
  - Randomly shuffle the dataset
    - Eliminate order impacts
  - For each example or each minibatch of examples until the whole dataset is exhausted
    - Update
  - Epoch
    - Each example in the whole dataset has been used once

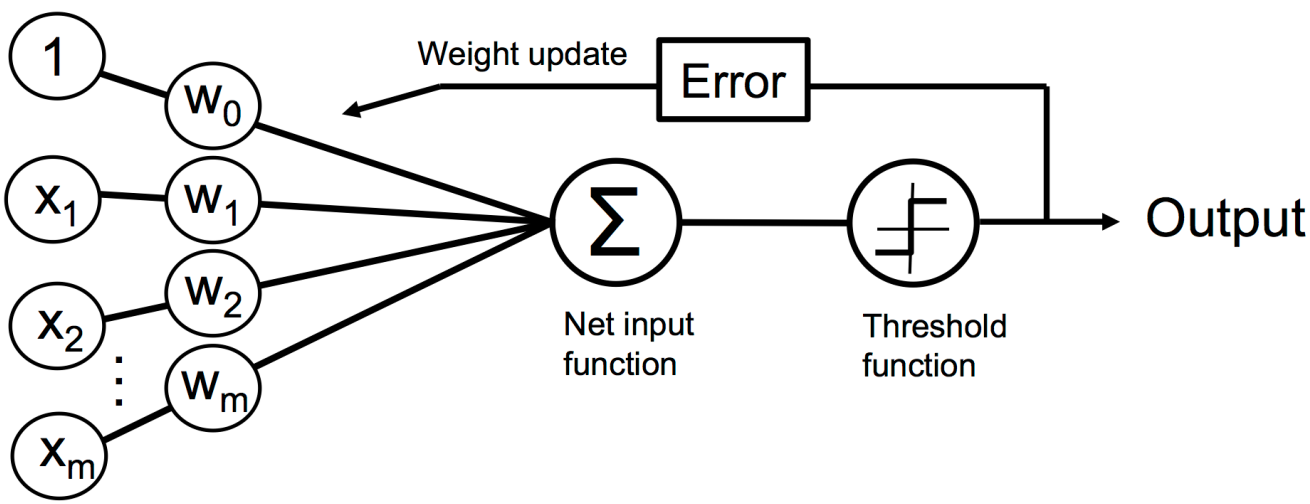
The Neuron Model

- McCulloch-Pitts (MCP) Neuron (1943)



Perceptron

- Rosenblatt 1957
- Each perceptron try to differentiate two classes
- Update sequentially with each example in an epoch

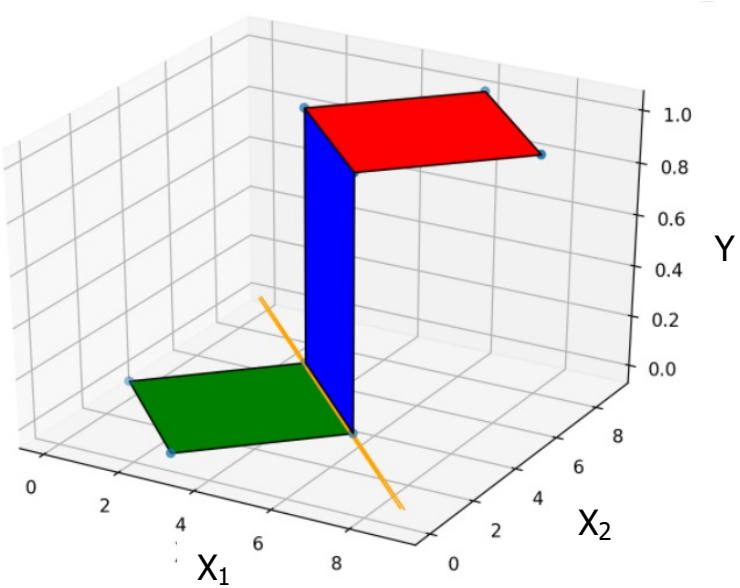


Perceptron Training

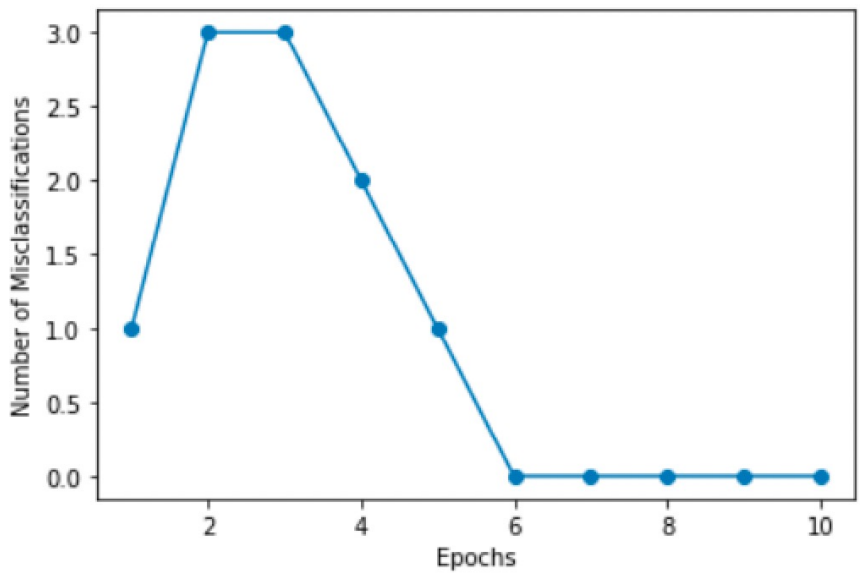
- Borrowed from linear regression

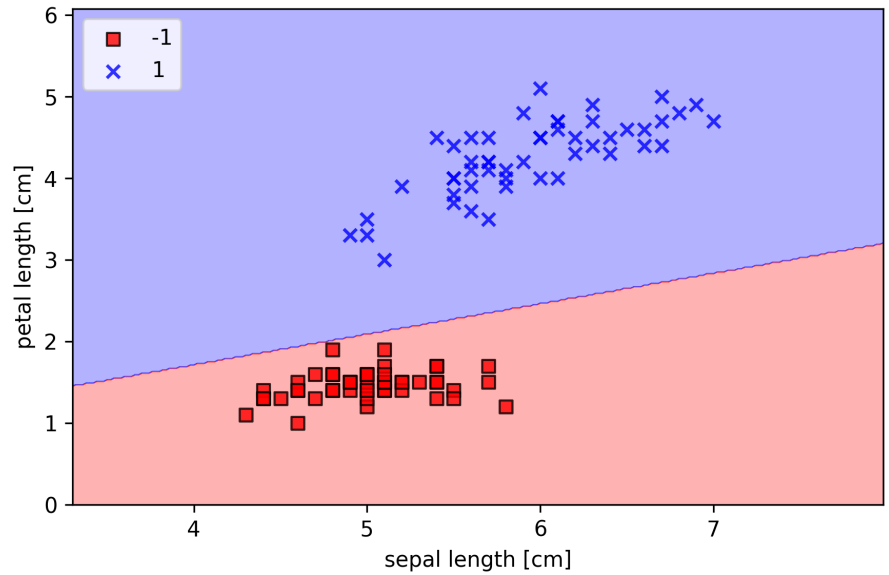
$$\Delta w_j = \epsilon\left(y^{(i)} - \hat{y}^{(i)}\right)x_j^{(i)}$$

$$w_j := w_j + \Delta w_j$$

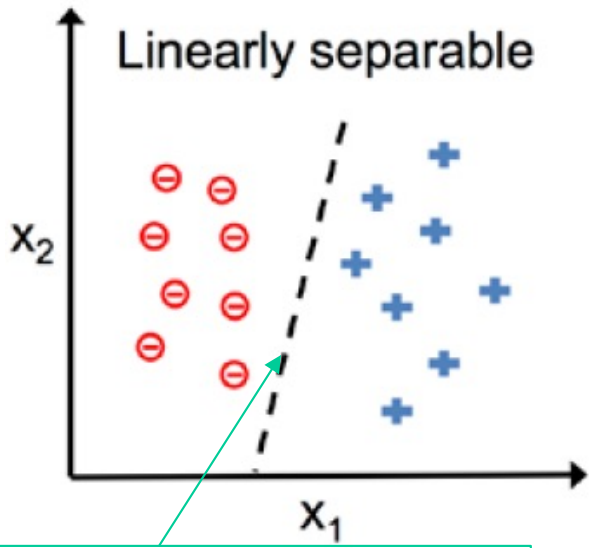


An Example

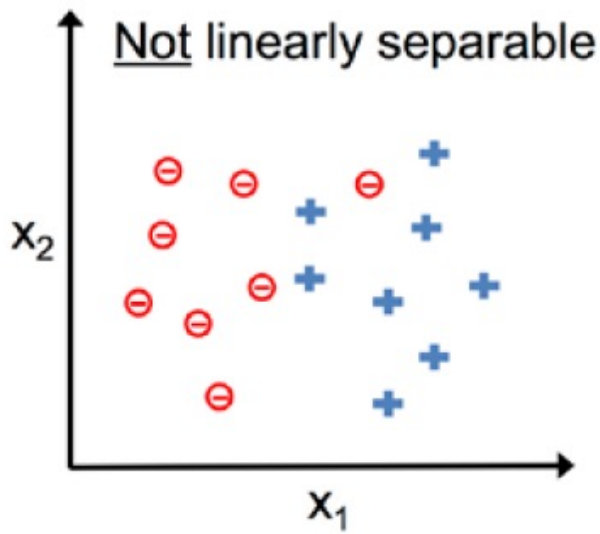

Number of Misclassifications
Epochs
3.0
2.5
2.0
1.5
1.0
0.5
0.0
2
4
6
8
10

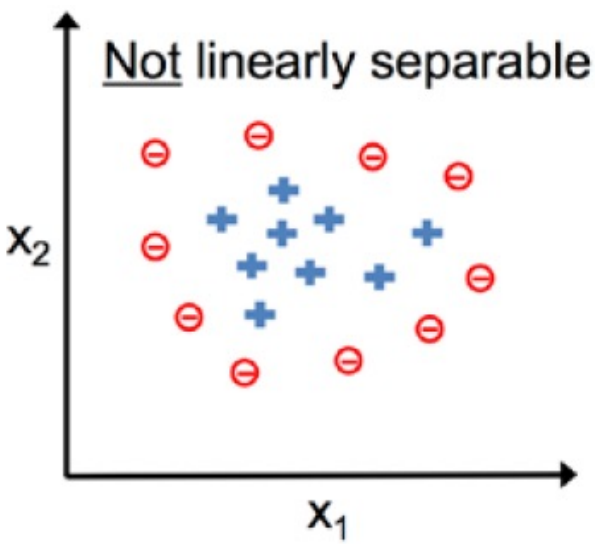

-1
1
petal length [cm]
sepal length [cm]
0
1
2
3
4
5
6
4
5
6
7

Perceptron Challenges



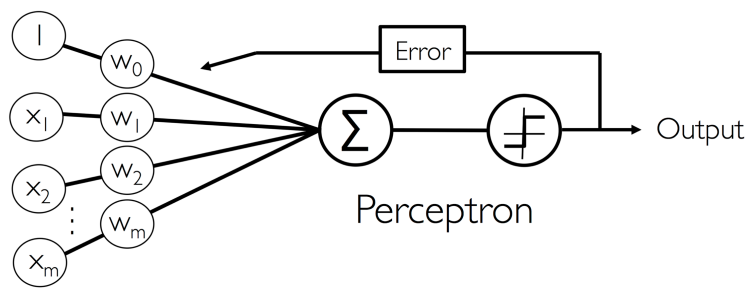
$$w_0 + w_1 X_1 + w_2 X_2 = 0$$

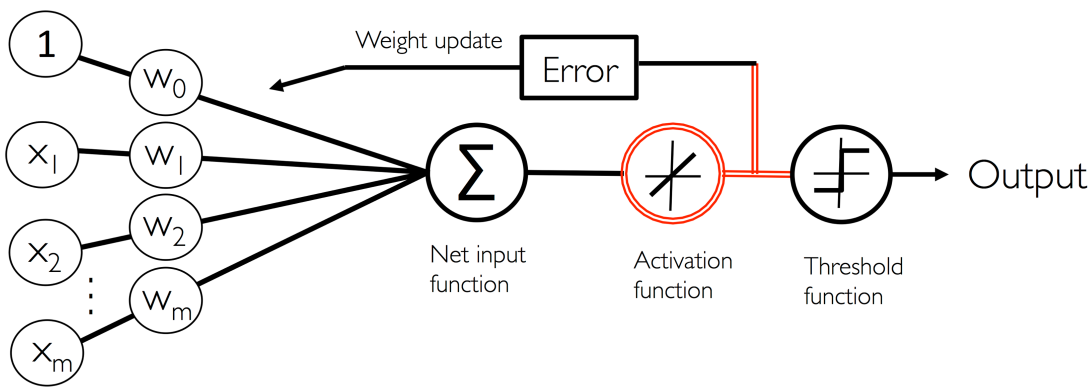




Adaline

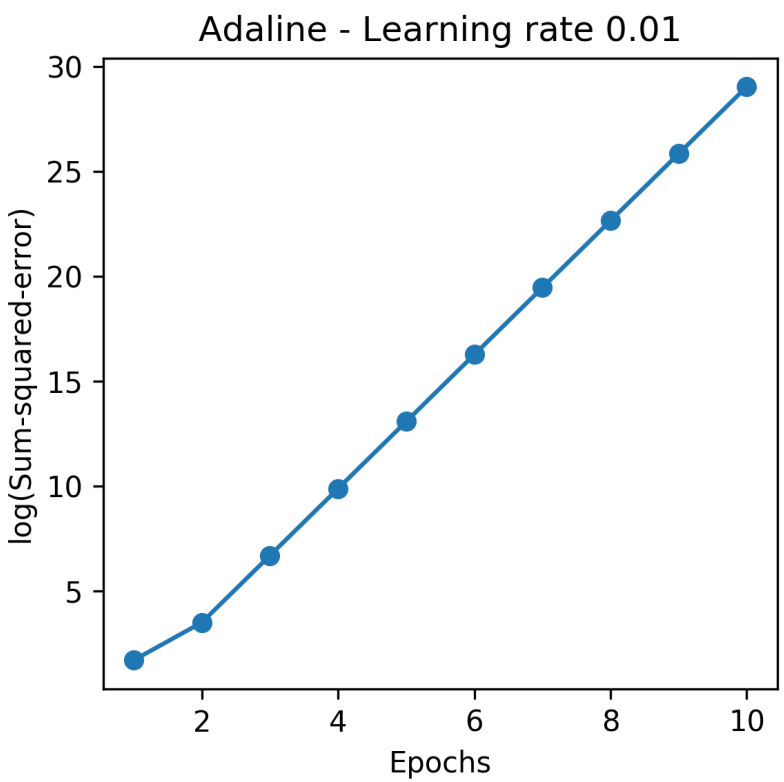
- ADAptive LInear Neuron
- Compare class labels with linear output
- Use batch update in each epoch

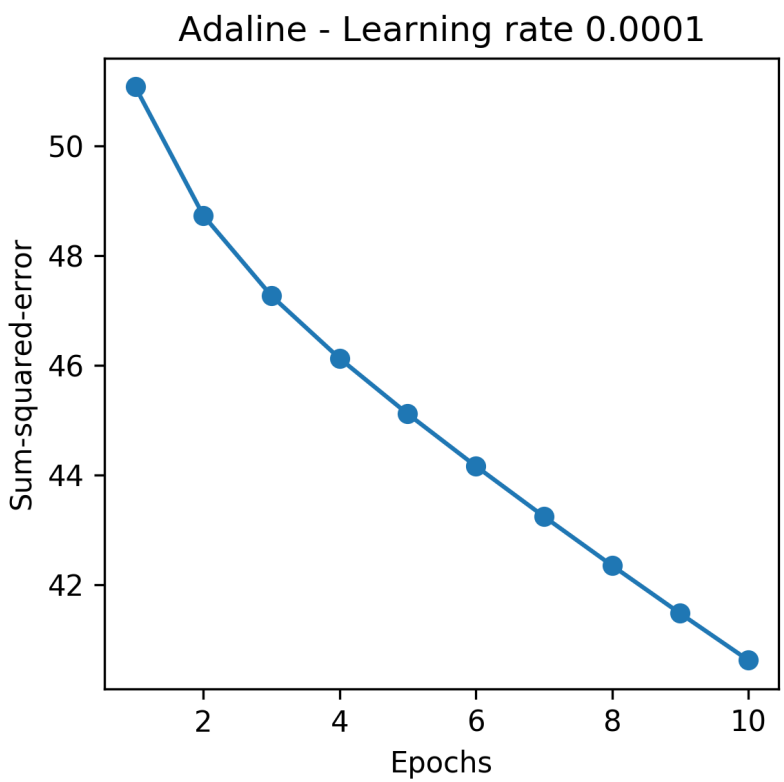




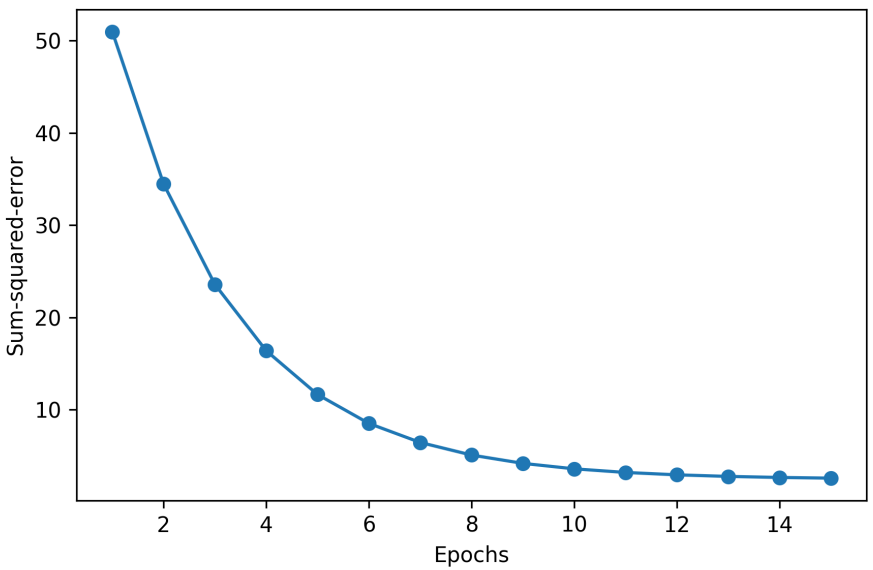
Adaptive Linear Neuron (Adaline)

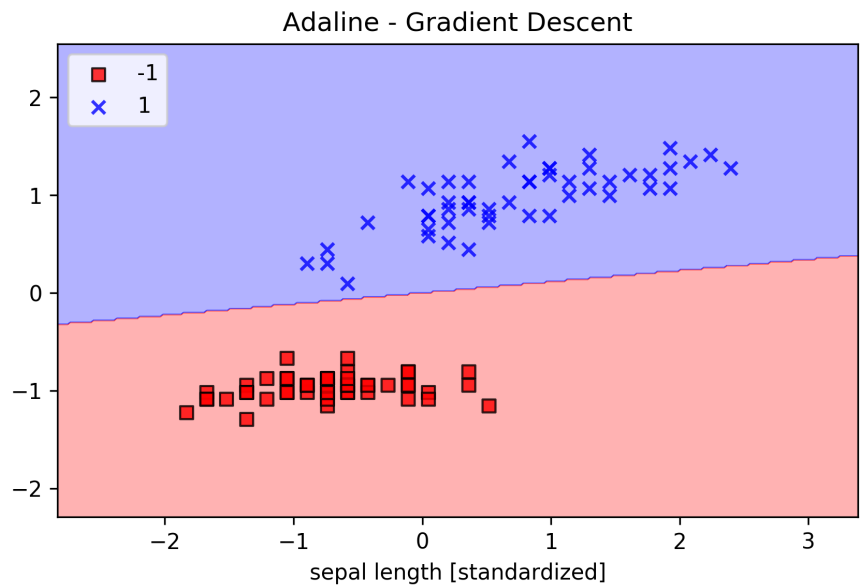
Example


Adaline - Learning rate 0.01
log(Sum-squared-error)
30
25
20
15
10
5
2
4
6
8
10
Epochs

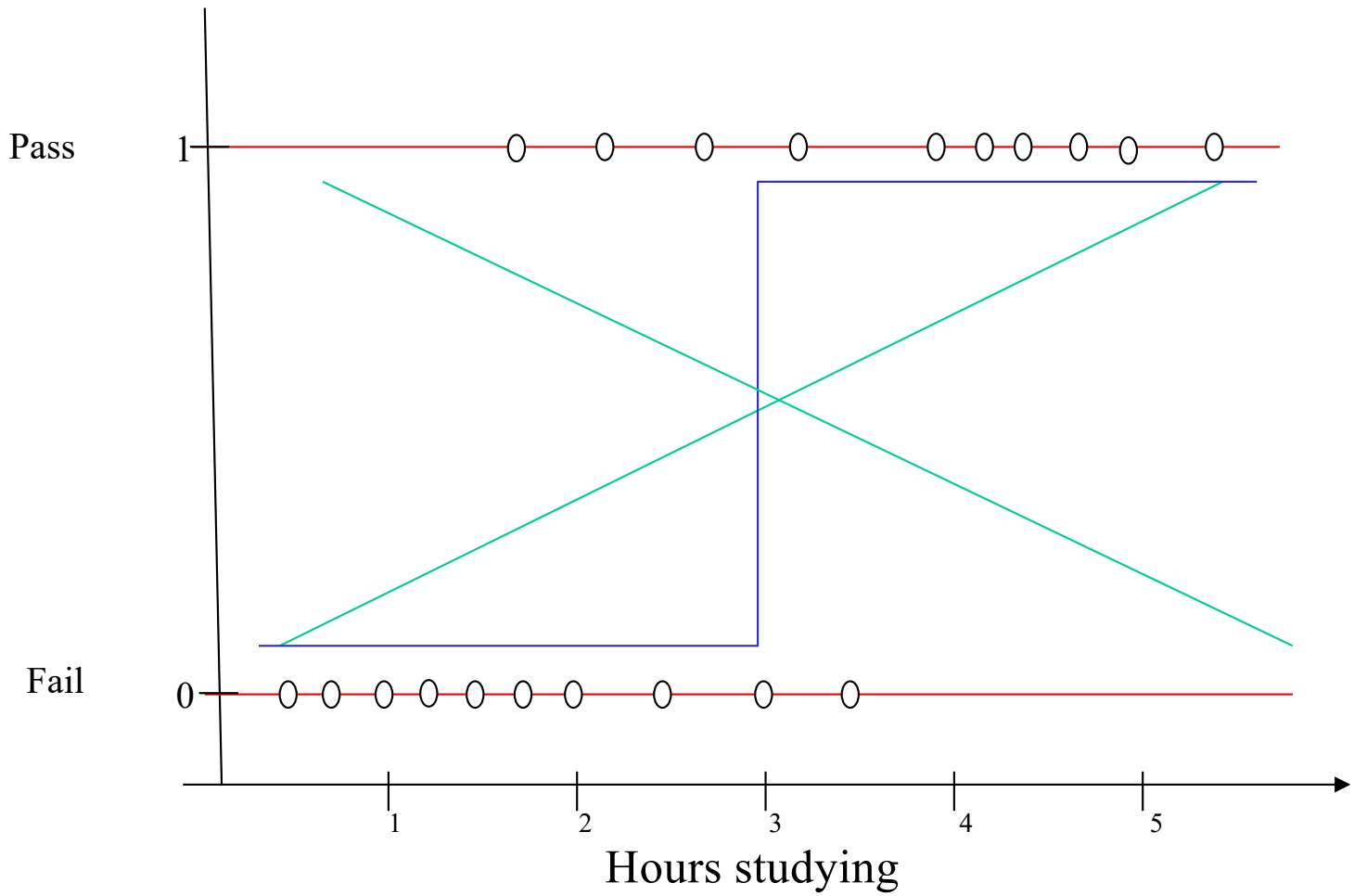

Adaline - Learning rate 0.0001
Sum-squared-error
50
48
46
44
42
2
4
6
8
10
Epochs

Example


Sum-squared-error
Epochs
50
40
30
20
10
2
4
6
8
10
12
14


Adaline - Gradient Descent
-1
1
sepal length [standardized]
2
1
0
−1
−2
−2
−1
0
1
2
3

Example: Student Passing Exam



Outline

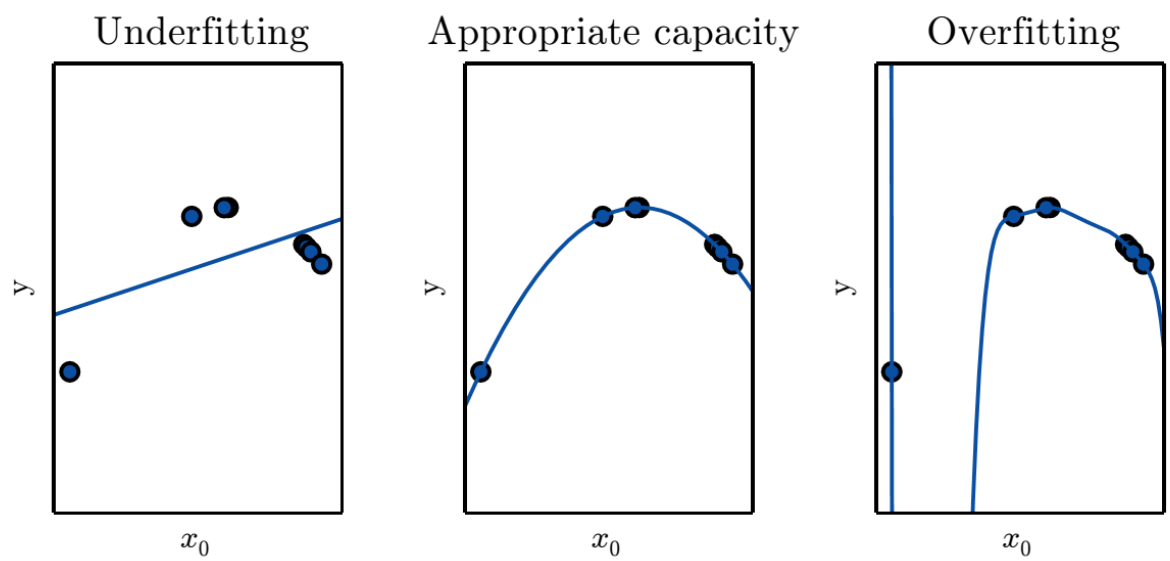
- Get started with models
- How to choose models

Errors

- Training error
  - Errors for training data
- Generalization error
  - Ability to perform well on new inputs
  - Estimated by using test data
- Goal of machine learning algorithm
  - Make the training error small
  - Reduce gap between test and training errors
- Underfitting vs. overfitting
  - Underfitting: training error not low enough
  - Overfitting: gap is too large

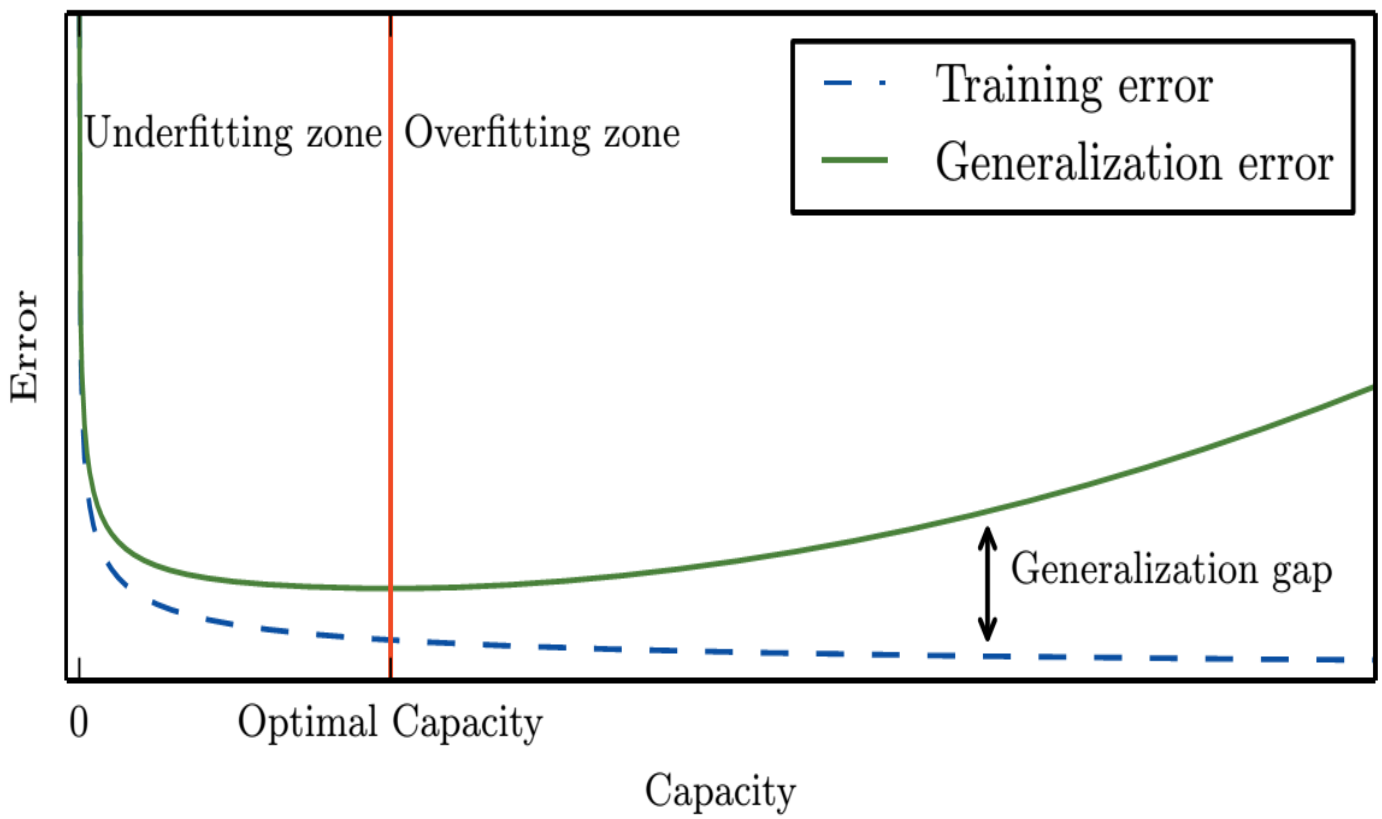
Capacity

- Capacity: ability to fit wide variety of functions
- Hypothesis space: the set of functions a model covers, e.g., linear regression to polynomials
  - Also called representational capacity
  - Limitation to find optimal function: effective capacity
- Ptolemy principle: choose simplest one when multiple available



Capacity vs. Errors

- Tradeoff between training error and generalization error



No Free Lunch Theorem

- No machine learning algorithm is universally better than any other
- A specific task requires specific machine learning
- Assumptions are made for each model family

Regularization

- Toward parsimony
  - A way to express preference
  - Ridge regression: shrink regression coefficients

$$L(\beta) = E(\beta) + \lambda\beta^T\beta = E(\beta) + \lambda\sum_{j=1}^{D}{\beta_j}^2$$

$$\hat{\beta} = (\boldsymbol{x}^T\boldsymbol{x} + \lambda\boldsymbol{I})^{-1}\boldsymbol{x}^T y$$

  - Lasso regression

$$\hat{\beta} = \underset{\beta}{\operatorname{argmin}}\left\{E(\beta) + \lambda\sum_{j=1}^{D}|\beta_j|\right\}$$

    - Solutions are nonlinear in $\boldsymbol{y}$
    - No closed form solution
    - Tend to make some coefficients zero
  - General regression

$$\hat{\beta} = \underset{\beta}{\operatorname{argmin}}\left\{E(\beta) + \lambda\sum_{j=1}^{D}|\beta_j|^q\right\}$$

Building Good Training Datasets

- Handling missing values
  - Eliminating training examples or features with missing values
  - Imputing missing values with interpolation
- Bring features onto the same scale
  - Normalization

$$x'_i = \frac{x_i - x_{min}}{x_{max} - x_{min}}$$

  - Standardization

$$x''_i = \frac{x_i - \mu_i}{\sigma_i}$$

- Dimension reduction
- Partitioning dataset into separate training, validation and test datasets

Summary

- Key concepts

  Models, Prior assumptions, Regularization, Capacity, Overfitting, Underfitting
- Key techniques
  - GD, SGD, Regularization
- Models
  - Linear regression, Perceptron, Adaline

References

- *Deep Learning*, I. Goodfellow, Y. Bengio, and A. Courville, MIT Press, 2016
- *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, T. Hastie, R. Tibshirani, and J. Friedman, Springer, Second Edition, 2008
- *Python Machine Learning,* S. Raschka and V. Mirjalili, Packt Publishing, 3rd Edition, 2019